



Valle del Sol

Integración de Arquitectura de Microservicios

Subdirección de Gestión de Emergencias · Plataforma de reporte ciudadano y alertas

Evaluación Parcial 3 — DSY1106

Desarrollo Fullstack III

Docente: Israel Villagra Riquelme

Integrantes: Matías Ampuero · Tomás Escobar

29 de junio de 2026

Foco de la presentación:

Pruebas unitarias, de integración y end-to-end sobre el business core, con persistencia e integración real.

EL CASO

El problema a resolver

Municipalidad Valle del Sol, Región San Jorge — extensas áreas forestales y viviendas rurales.

Dolores detectados

- Detección tardía: 35 min promedio hasta el reporte.
- Solo el 22% de los reportes provienen de vecinos.
- Coordinación informal y registros en papel (sin GIS).
- 73% de la comunidad: las alertas llegan tarde.

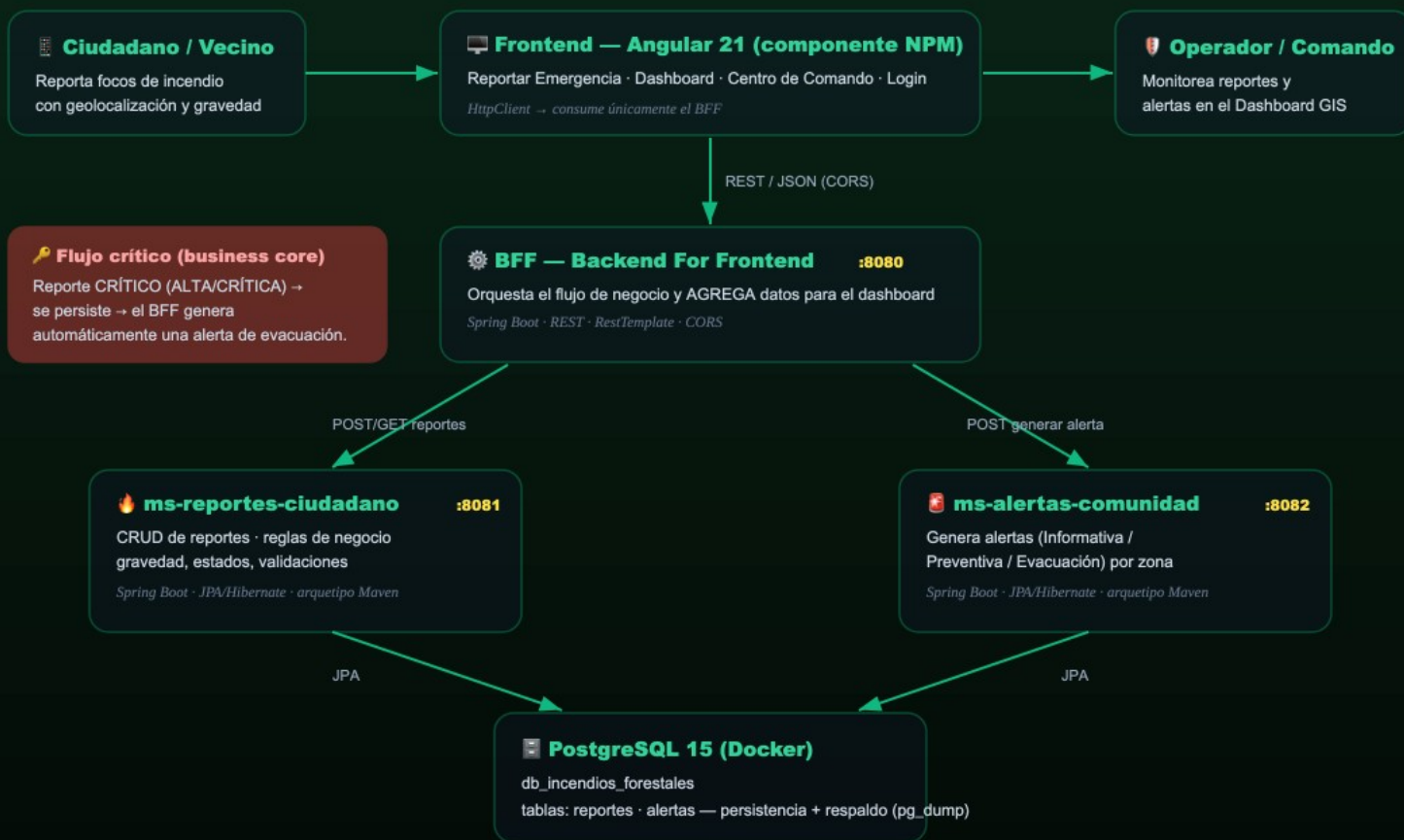
Oportunidad

- ✓ Reducir 15 min la detección → -40% superficie afectada.
- ✓ Pasar de respuesta reactiva a prevención proactiva.
- ✓ Reporte ciudadano con geolocalización.
- ✓ Motor de alertas masivas a la comunidad.

Arquitectura de microservicios

Valle del Sol — Arquitectura de Microservicios

Subdirección de Gestión de Emergencias · Plataforma de reporte ciudadano y alertas comunitarias (EP3 · DSY1106)



Componentes

- Frontend Angular (NPM)
- BFF (orquesta + agrega)
- ms-reportes-ciudadano
- ms-alertas-comunidad
- PostgreSQL (Docker)
- REST/JSON entre todos
- JPA para persistencia

Flujo de negocio crítico



Reporte CRÍTICO (ALTA/CRÍTICA) → se persiste → el BFF genera automáticamente una alerta de EVACUACIÓN.

Integración y persistencia logradas

Integración REST

- Frontend → BFF → microservicios.
- El BFF orquesta y agrega.
- CORS habilitado para el front.

Persistencia

- PostgreSQL 15 en Docker.
- JPA/Hibernate en cada MS.
- Tablas reportes y alertas.

Componentes

- Angular 21 (NPM).
- 3 servicios Spring Boot.
- Arquetipos Maven, GitHub.

Estrategia: pirámide de pruebas

Unitarias (17)

Reglas de negocio aisladas

JUnit 5 · Mockito · AssertJ

Integración (8)

REST → Servicio → Repositorio → BD

Spring Boot Test · MockMvc · H2

End-to-End (2)

Flujo completo a través del BFF

MockRestServiceServer

E2E manual

Sistema real en ejecución

curl · Scripts/prueba-e2e.sh

RESULTADOS

Resultados de las pruebas

27

pruebas en verde (0 fallidas)

17 · 8 · 2

unitarias · integración · e2e

✓ **Persistencia real**

verificada en PostgreSQL

Microservicio	Unit	Integr.	E2E	Cob. instr.	Cob. ramas
ms-reportes-ciudadano	8	5	—	73,9%	63,6%
ms-alertas-comunidad	6	3	—	79,0%	68,0%
bff-valle-sol	3	—	2	68,8%	39,3%

Bug detectado por las pruebas (BUG-01)

Síntoma: el sistema aceptaba coordenadas fuera de rango (lat=200), generando un punto inválido en el mapa GIS.

Causa: no se validaba el rango geográfico de las coordenadas.

Corrección: método validarCoordenadas() + validación @DecimalMin/@DecimalMax. Ahora responde HTTP 400 y no persiste.

Detectado por: CP-U03 · CP-U04 · CP-I02.

Otras mejoras a partir de las pruebas:

- Descripción mínima de 10 caracteres (utilidad para las brigadas).
- Máquina de estados que rechaza transiciones ilegales (ej. RESUELTO → PENDIENTE).
- Manejo de microservicio caído en el BFF: HTTP 503 controlado en vez de error 500.

Demostración y evidencia

```
$ ./Scripts/prueba-e2e.sh
```

```
[2] Reporte CRÍTICO → alertaGenerada:true · nivel:EVACUACION PASA
```

```
[3] Reporte MEDIA → alertaGenerada:false PASA
```

```
[4] Coordenada inválida (lat=200) → HTTP 400 PASA
```

RESULTADO: 7 pruebas OK, 0 fallidas

1. Levantar: `docker compose up -d + Scripts/levantar-local.sh + ng serve`.
2. Frontend: el vecino envía un reporte crítico → recibe folio + alerta de evacuación.
3. Dashboard / Centro de Comando muestra reportes y alertas en vivo desde PostgreSQL.
4. Reporte de cobertura JaCoCo en `*/target/site/jacoco/index.html`.

Conclusiones

- ✓ Integración real de frontend y backend con comunicación REST y persistencia en PostgreSQL.
- ✓ El proceso de negocio crítico quedó cubierto en los tres niveles de prueba + verificación manual.
- ✓ Las pruebas no se limitaron al camino feliz: detectaron y corrigieron el BUG-01 y endurecieron validaciones.
- ✓ 27 pruebas en verde; cobertura concentrada en la lógica de negocio (donde está el riesgo).
- ✓ Solución ejecutable, demostrable y versionada en GitHub.

¿Preguntas?